

Sistemas Operacionais e Computação em Nuvem

Entrega 2

Grupo: Enrolados

Guilherme Barioni
Iury Xavier da Silva Mangueira
Lilian Mercedes Paye Conde
Marcus Miranda Duque

Preparando o sistema

Conectamos diretamente com a maquina virtual na azure que criamos na entrega anterior usando ssh:

```
[iury@Arch ~]$ ssh enrolados@[REDACTED]
Linux DebianLinux 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Apr 29 14:20:13 2026 from [REDACTED]
enrolados@DebianLinux:~$
```

Para rodar os codigos de monitoramento e IA, instalamos o python, o pip(gerenciador de pacotes do python), o venv para criar um ambiente virtual isolado para a instalações de dependências, o htop e o sysstat para monitorar a performance do sistema:

```
enrolados@DebianLinux:~$ sudo apt install python3 python3-pip python3-venv htop sysstat
python3 já é a versão mais nova (3.13.5-1).
htop já é a versão mais nova (3.4.1-5).
Instalando:
  python3-pip  python3-venv  sysstat

Instalando dependências:
  binutils          g++-14-x86-64-linux-gnu  libalgorithm-merge-perl  libctf0          libitm1          libubsan1          python3-wheel
  binutils-common  g++-x86-64-linux-gnu    libasan8                 libdpkg-perl     liblsan0          linux-libc-dev     python3.13-dev
  binutils-x86-64-linux-gnu  gcc                    libbinutils              libexpat1-dev    libpython3-dev    make               python3.13-venv
  build-essential  gcc-14                 libc-dev-bin             libfakeroot      libpython3.13-dev  manpages-dev       rpcsvc-proto
  dpkg-dev          gcc-14-x86-64-linux-gnu  libc6-dev                libfile-fcntllock-perl  libquadauth0     python3-dev        zlib1g-dev
  fakeroot          gcc-x86-64-linux-gnu    libcc1-0                 libgcc-14-dev    libquadmath0      python3-packaging   python3-pip-whl
  g++               libalgorithm-diff-perl  libcrypt-dev             libgprofng0      librsync0          python3-pip-whl     python3-setuptools-whl
  g++-14            libalgorithm-diff-xs-perl  libctf-nobfd0            libhwasan0       libstdc++-14-dev  python3-setuptools-whl

Pacotes sugeridos:
  binutils-doc  debian-keyring  g++-14-multilib  autoconf  flex  gcc-doc  gdb-x86-64-linux-gnu  bzip2  libstdc++-14-doc  python3-setuptools
  gprofng-gui  debian-tag2upload-keyring  gcc-14-doc  automake  bison  gcc-14-multilib  gdb  libstdc++-14-dev  isag
  binutils-gold  g++-multilib  gcc-multilib  libtool  gdb  gcc-14-locales  glibc-doc  make-doc

Resumo:
  Atualizando: 0, Instalando: 56, Removendo: 0, Não atualizando: 0
  Tamanho de download: 75,4 MB
  Espaço necessário: 294 MB / 4.949 MB disponível
```

Criamos uma pasta chamada monitoramento_ia_nuvem para armazenar os scripts:

```
enrolados@DebianLinux:~/Documentos$ mkdir monitoramento_ia_nuvem
enrolados@DebianLinux:~/Documentos$ cd monitoramento_ia_nuvem/
```

Criamos o ambiente virtual isolado e ativamos o venv:

```
enrolados@DebianLinux:~/Documentos/monitoramento_ia_nuvem$ python3 -m venv venv
enrolados@DebianLinux:~/Documentos/monitoramento_ia_nuvem$ source venv/bin/activate
(venv) enrolados@DebianLinux:~/Documentos/monitoramento_ia_nuvem$
```

Dentro do ambiente virtual instalamos as dependências necessárias para o funcionamento dos scripts python:

```
(venv) enrolados@DebianLinux:~/Documentos/monitoramento_ia_nuvem$ pip install psutil scikit-learn pandas matplotlib seaborn jupyter
```

O psutil sendo utilizado para monitorar os recursos do sistema, scikit-learn é utilizado no treinamento e previsão com IAs, pandas, matplotlib e seaborn são utilizados na criação de dataframes e no plot de graficos.

Criamos o script coletor. que é responsável pelo monitoramento do sistema, ele coleta as informações sobre o uso de CPU e memória e armazena os dados em uma planilha csv.

```
GNU nano 8.4                                                                    coletor.py
import psutil
import csv
import time
from datetime import datetime

# Nome do arquivo de saída
ARQUIVO_CSV = 'dados_monitoramento.csv'

# Criar o arquivo e escrever o cabeçalho
with open(ARQUIVO_CSV, mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['DataHora', 'Tempo_Segundos', 'CPU_Percentual', 'Memoria_Percentual'])

print(f"Iniciando coleta de dados em {ARQUIVO_CSV}. Pressione Ctrl+C para parar.")

tempo_inicial = time.time()

try:
    while True:
        agora = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        tempo_decorrido = int(time.time() - tempo_inicial)

        # Coletar uso de CPU e Memória
        uso_cpu = psutil.cpu_percent(interval=None) # interval=None para não travar o loop
        uso_memoria = psutil.virtual_memory().percent

        # Salvar no CSV
        with open(ARQUIVO_CSV, mode='a', newline='') as file:
            writer = csv.writer(file)
            writer.writerow([agora, tempo_decorrido, uso_cpu, uso_memoria])

        # Aguardar 2 segundos para a próxima coleta
        time.sleep(2)

except KeyboardInterrupt:
    print("\nColeta de dados finalizada pelo usuário.")
```

Deixamos o script de monitoramento rodar por 2 minutos com a VM ociosa para comparar os dados com ela sobre estresse.

Para coletar dados da VM sobre estresse, criamos um script que busca forçar a CPU a realizar uma alta quantidade de cálculos rapidamente, além da criação de listas enormes para consumir muito mais memória RAM:

```

import time
import multiprocessing

def estressar_cpu():
    """Cria um loop infinito com cálculos matemáticos pesados para forçar a CPU."""
    while True:
        _ = [x**2 for x in range(10000)]

def estressar_memoria():
    """Cria uma lista gigante para consumir a memória RAM gradativamente."""
    dados_pesados = []
    while True:
        dados_pesados.append('A' * 10**6) # Adiciona 1MB de strings por iteração
        time.sleep(0.1) # Pausa leve para não travar a VM instantaneamente

if __name__ == '__main__':
    print("Iniciando Teste de Estresse! Pressione Ctrl+C para abortar.")
    print("Aviso: Sua VM pode ficar lenta.")

    # Inicia processos separados para estressar CPU e Memória simultaneamente
    processo_cpu = multiprocessing.Process(target=estressar_cpu)
    processo_mem = multiprocessing.Process(target=estressar_memoria)

    processo_cpu.start()
    processo_mem.start()

    try:
        # Deixa rodando por tempo indeterminado até você apertar Ctrl+C
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("\nFinalizando processos de estresse...")
        processo_cpu.terminate()
        processo_mem.terminate()
        print("Sistema normalizado.")

```

Para realizar a análise dos dados com modelos de IA, criamos um script chamado `analise_ia.py` utilizando três diferentes tipos de algoritmos, regressão linear para analisar tendências de CPU, k-means para agrupar os estados do servidor e Isolation Forests para detecção de anomalias:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.ensemble import IsolationForest

# Configuração visual dos gráficos
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)

# 1. CARREGAR OS DADOS
df = pd.read_csv('dados_monitoramento.csv')

# =====
# IA 1: REGRESSÃO LINEAR (Tendência da CPU)
# =====
# Objetivo: Identificar a tendência de uso da CPU ao longo do tempo.
X_reg = df[['Tempo_Segundos']]
y_reg = df['CPU_Percentual']

modelo_regressao = LinearRegression()
modelo_regressao.fit(X_reg, y_reg)
df['Tendencia_CPU'] = modelo_regressao.predict(X_reg)

plt.figure()
plt.plot(df['Tempo_Segundos'], df['CPU_Percentual'], label='Uso Real de CPU', color='blue', alpha=0.6)
plt.plot(df['Tempo_Segundos'], df['Tendencia_CPU'], label='Tendência (Regressão)', color='red', linewidth=2)
plt.title('Regressão Linear: Tendência de Uso da CPU ao Longo do Tempo')
plt.xlabel('Tempo (Segundos)')
plt.ylabel('Uso da CPU (%)')
plt.legend()
plt.show()

```

```

# =====
# IA 2: CLUSTERIZAÇÃO (K-Means)
# =====
# Objetivo: Agrupar os estados do servidor (ex: Ocioso, Normal, Sobrecarga).
# Usaremos CPU e Memória como variáveis.
X_cluster = df[['CPU_Percentual', 'Memoria_Percentual']]

# Definimos 3 clusters (estados)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Estado_Cluster'] = kmeans.fit_predict(X_cluster)

plt.figure()
sns.scatterplot(data=df, x='CPU_Percentual', y='Memoria_Percentual', hue='Estado_Cluster', palette='viridis', s=100)
plt.title('K-Means: Agrupamento de Estados do Servidor')
plt.xlabel('Uso da CPU (%)')
plt.ylabel('Uso da Memória (%)')
plt.legend(title='Clusters')
plt.show()

# =====
# IA 3: DETECÇÃO DE ANOMALIAS (Isolation Forest)
# =====
# Objetivo: Detectar comportamentos anômalos (como os picos do teste de estresse).
# contamination=0.1 significa que assumimos que ~10% dos dados são anomalias (picos).
iso_forest = IsolationForest(contamination=0.1, random_state=42)
df['Anomalia'] = iso_forest.fit_predict(X_cluster)

# O algoritmo retorna -1 para anomalias e 1 para dados normais.
df['Status_Anomalia'] = df['Anomalia'].apply(lambda x: 'Anomalia' if x == -1 else 'Normal')

plt.figure()
sns.scatterplot(data=df, x='Tempo_Segundos', y='CPU_Percentual', hue='Status_Anomalia',
                palette={'Normal': 'green', 'Anomalia': 'red'}, s=80)
plt.title('Isolation Forest: Detecção de Picos Anômalos na CPU')
plt.xlabel('Tempo (Segundos)')
plt.ylabel('Uso da CPU (%)')
plt.show()

# Visualização final para o Relatório: Análise Conjunta no Tempo
plt.figure()
plt.plot(df['Tempo_Segundos'], df['CPU_Percentual'], label='CPU (%)', color='blue')
plt.plot(df['Tempo_Segundos'], df['Memoria_Percentual'], label='Memória (%)', color='orange')

# Destacar as anomalias detectadas no gráfico temporal
anomalias = df[df['Anomalia'] == -1]
plt.scatter(anomalias['Tempo_Segundos'], anomalias['CPU_Percentual'], color='red', label='Anomalia Detectada', zorder=5)

plt.title('Monitoramento Completo com Destaque de Anomalias')
plt.xlabel('Tempo (Segundos)')
plt.ylabel('Uso (%)')
plt.legend()
plt.show()

```

O código também exibe os graficos resultantes para obtermos uma visualização mais complexa.

Aplicação da IA

A aplicação de Inteligência Artificial neste projeto demonstrou como algoritmos de Machine Learning podem transformar dados brutos de telemetria em insights acionáveis para o gerenciamento de infraestrutura em nuvem. O objetivo foi demonstrar como modelos de Machine Learning podem processar a telemetria do sistema operacional em tempo real, transformando dados brutos em decisões arquitetônicas. A estratégia de IA foi desenhada em três camadas fundamentais: Previsibilidade, Otimização e Resiliência. A abordagem foi dividida em três pilares principais:

Análise de Tendência (Supervisionada - Regressão Linear)

O algoritmo traçou a linha de melhor ajuste para o consumo de CPU ao longo da linha do tempo, calculando o coeficiente angular da demanda computacional. Em cenários reais de Computação em Nuvem, esse tipo de modelo é fundamental para o *Auto Scaling* preditivo. Ao identificar que a tendência de uso da CPU está em ascensão antes mesmo de atingir o limite crítico, o sistema pode provisionar novas máquinas virtuais proativamente, garantindo alta disponibilidade.

Identificação de Perfis (Não-Supervisionada – K-Means)

Sem rotular os dados previamente, deixamos o algoritmo *K-Means* analisar o padrão geométrico da relação entre CPU e RAM. Essa técnica permite classificar a "saúde" do servidor

dinamicamente. Em um ambiente corporativo, isso poderia ser usado para descobrir que certos servidores possuem um perfil de "Alto uso de RAM e Baixo uso de CPU", o que indicaria que a nuvem deve redimensionar as instâncias para famílias otimizadas para memória (como a série E na Azure), otimizando custos e performance.

Segurança e Resiliência (Não-Supervisionada - Isolation Forest)

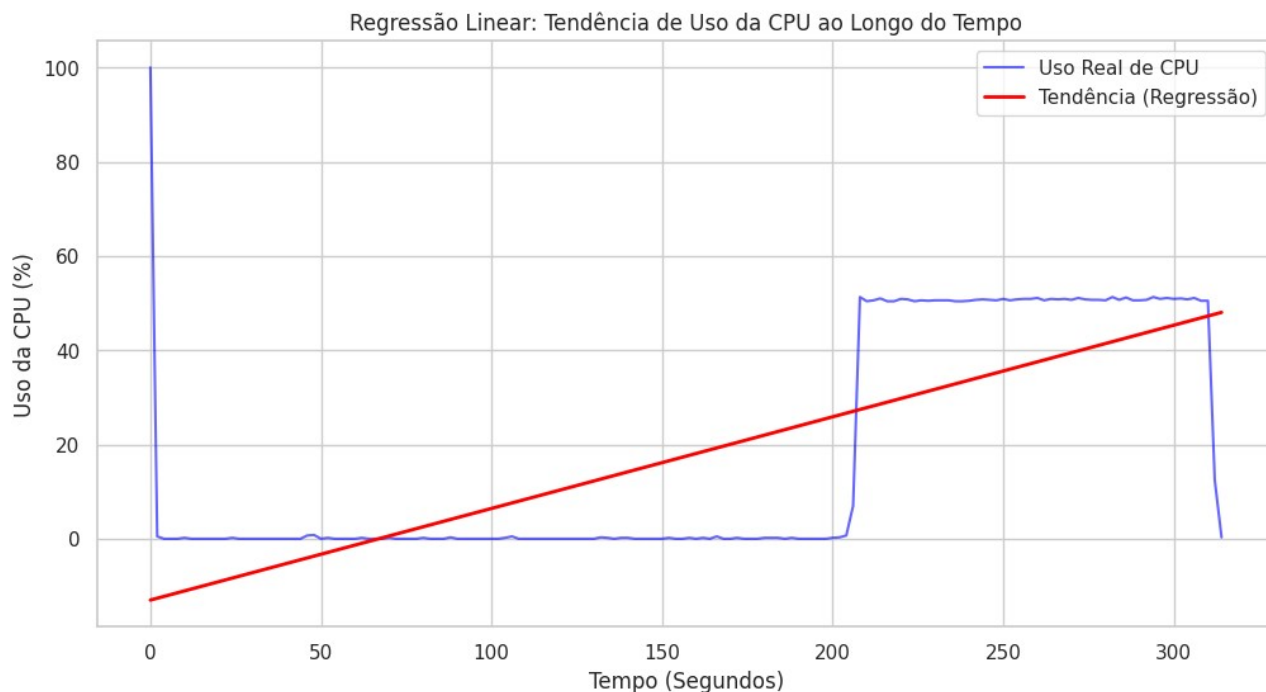
Configuramos o algoritmo assumindo uma contaminação de 10% (esperando que 10% dos dados fossem anômalos) para varrer todo o registro coletados do servidor em busca de comportamentos discrepantes. Em sistemas de produção, saltos instantâneos de consumo geralmente indicam problemas graves, como ataques de negação de serviço (DDoS), falhas catastróficas em aplicações (loops infinitos) ou invasões de cryptojacking. O Isolation Forest atua como um sentinela que poderia disparar alertas imediatos para as equipes de SRE (Site Reliability Engineering) assim que o padrão de uso sofresse uma ruptura inexplicável.

Análise e Interpretação dos Resultados

Os gráficos gerados pelo script `analise_ia.py` nos fornecem uma visão clara de como o sistema se comportou durante os períodos de ociosidade e sob o teste de estresse, validando a eficácia dos modelos de Inteligência Artificial escolhidos.

Regressão Linear: Tendência de Uso da CPU

O gráfico de Regressão Linear ilustra a variação da CPU ao longo do tempo. A linha azul mostra o comportamento real: um pico isolado inicial (provavelmente durante a inicialização do monitoramento), seguido por um longo período em quase 0% (VM ociosa) e, após cerca de 200 segundos, um salto abrupto e constante para a faixa dos 50% de uso. A linha vermelha representa o modelo de regressão linear, que traça uma reta ascendente. Isso indica que, estatisticamente, a tendência do servidor ao longo do tempo analisado foi de aumento contínuo de carga, refletindo a transição do estado ocioso para o estado de estresse.



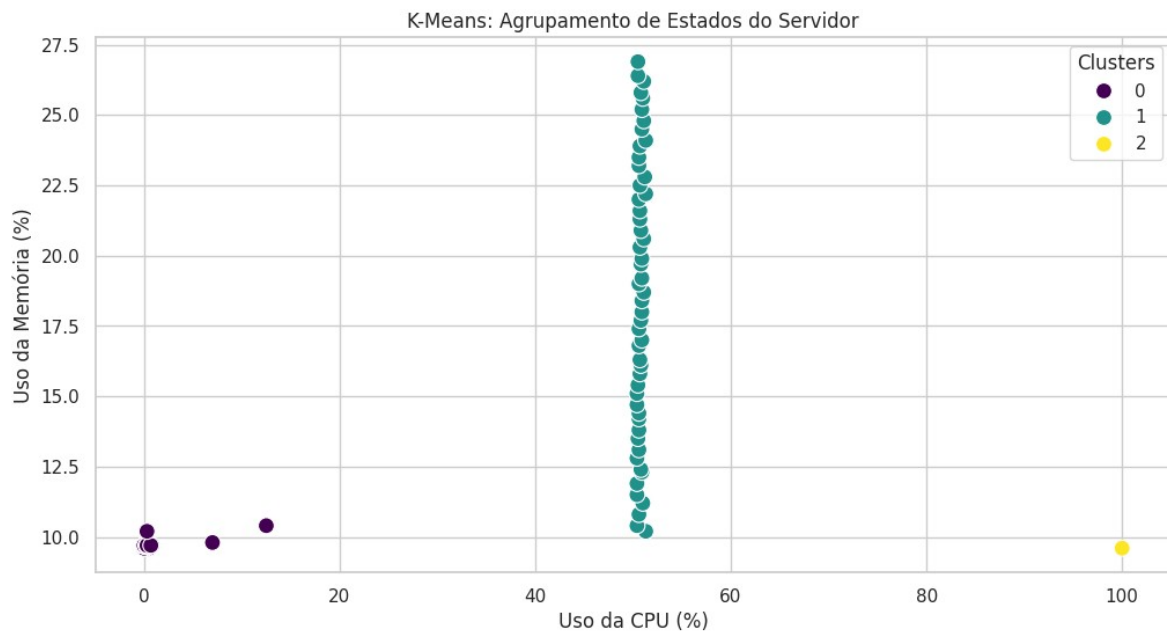
K-Means: Agrupamento de Estados do Servidor

O modelo K-Means foi configurado para encontrar 3 clusters (grupos) baseados no uso de CPU e Memória. O gráfico de dispersão revela a criação de três perfis comportamentais perfeitos:

Cluster 0 (Roxo): Representa o estado "Ocioso". Concentra-se em CPU próxima a 0% e uso de memória em cerca de 10%

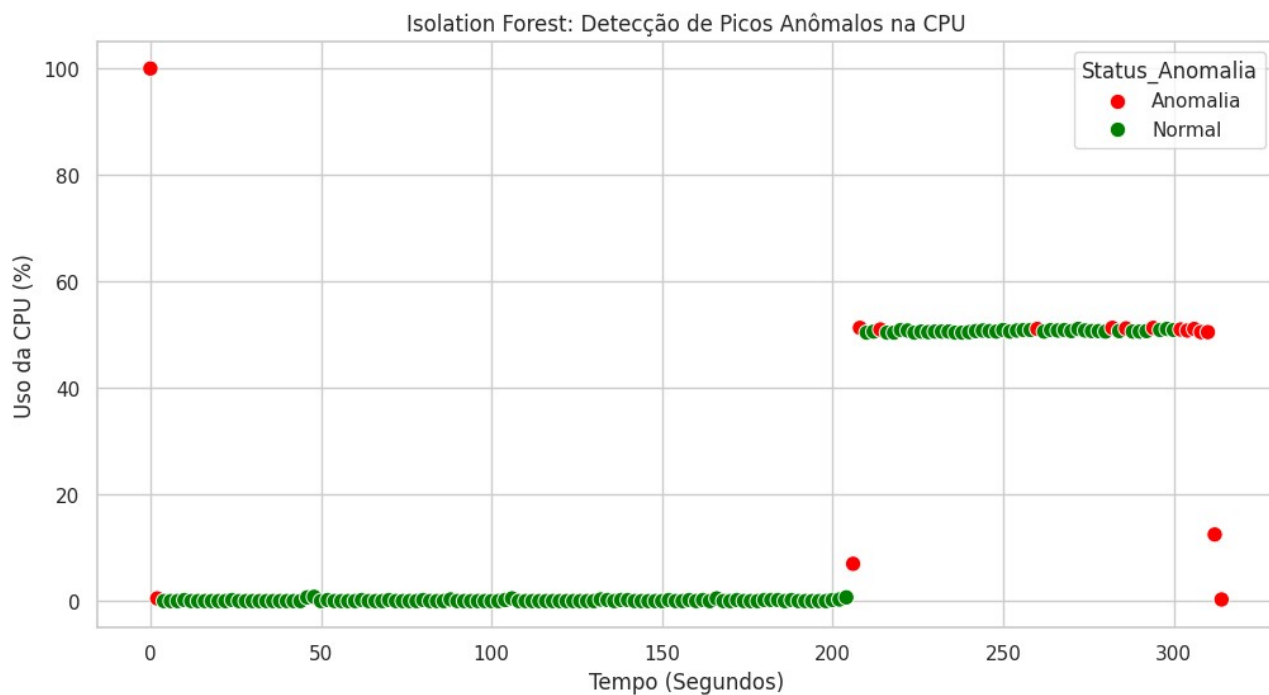
Cluster 1 (Verde-azulado): Representa o estado de "Sobrecarga/Estresse". A CPU está travada em aproximadamente 50%, mas notamos que os pontos sobem verticalmente no eixo Y. Isso comprova o funcionamento da função `estressar_memoria()`, que consumiu a RAM gradativamente (indo de ~10% a quase 27%) enquanto o processo de CPU rodava em paralelo.

Cluster 2 (Amarelo): Representa o estado de "Pico Máximo", isolando um momento específico em que a CPU bateu 100%, mas a memória ainda estava baixa (~10%).



Isolation Forest: Detecção de Anomalias

O gráfico do *Isolation Forest* demonstra a capacidade do modelo de identificar eventos fora do padrão estatístico (pontos vermelhos). O modelo foi extremamente preciso: ele não marcou o uso contínuo de 50% de CPU como anomalia após o sistema estabilizar nesse patamar, mas sim as transições bruscas de estado. As anomalias detectadas foram o pico de 100% no início, o exato segundo em que o teste de estresse foi iniciado (salto de 0% para 50%) e o momento em que o teste foi abortado pelo usuário (queda abrupta de 50% para 0%).



Conclusão

A integração de scripts simples em Python com algoritmos consagrados de Machine Learning provou ser uma solução robusta e de baixo custo. A máquina virtual Linux no Debian respondeu conforme o esperado aos estresses introduzidos, e os modelos não apenas registraram o ocorrido, mas foram capazes de entender matematicamente as fases do ciclo de vida do servidor. Concluimos que a Inteligência Artificial automatiza e eleva a precisão do monitoramento de sistemas operacionais, passando de uma postura reativa (olhar para os gráficos após a queda) para uma postura analítica e preditiva.